

Module 0: Environment setup and code orientation

Instructional time. 20 min reading. Setup: 30 to 60 min for the environment.

1. Overview

This module prepares a working machine for the four analysis notebooks and introduces the way the course code is written. Everything in the course is Python. You will install one pinned environment, start JupyterLab in a browser, confirm the setup, and read enough of the code structure to know where any given calculation lives. Module 1 begins with discovery and download of the data itself.

No prior single-cell or genomics experience is assumed. You should be comfortable opening a terminal, running a command, and editing a text file. If you have written Python at the level of loops, functions and a pandas DataFrame, you have enough to follow every pattern in the second half of this document.

By the end you will have:

- A conda environment named `CFDE_lung_env` running Python 3.11 with 27 pinned packages
- JupyterLab open in a browser with that environment selected as the kernel
- A verified setup, checked by a script you run before you start Module 1
- Disk and network budgeted for the 814.87 MB Module 1 downloads, itemized in Appendix B
- A map of the code: which directory holds the analysis, which holds the orchestration, and how the two are joined by one configuration file

2. Before you begin

Requirement	Value
Operating system	macOS or Linux. Windows through WSL2, where every command below is unchanged
Python	3.11, installed by the environment. You do not install Python separately
RAM	16 GB recommended. Peak measured use is 7.6 GB during the Module 2 differential expression step, so 8 GB is the minimum
Free disk	5 GB. The data is about 814.87 MB and the notebooks write processed objects beside it
Network	The download is about 814.87 MB, roughly 2 minutes at 50 Mbps and 11 minutes at 10 Mbps
Accounts	None. Every file the course uses is openly accessible

Nothing in this course requires a cluster, a GPU, or an institutional login.

3. Installing Python and the course environment

The course pins 27 packages to exact versions. Pinning is not bureaucracy here: `scanpy`, `harmonypy` and `pydeseq2` change clustering assignments and differential expression results between minor versions, so an

unpinned environment will not reproduce the numbers printed in the module documents. Install the environment as written and the results match.

1. Install Miniforge

Miniforge is a minimal conda distribution that installs packages from conda-forge by default and ships the `mamba` solver. Download the installer for your platform from <https://github.com/conda-forge/miniforge> and run it, then close and reopen the terminal.

Confirm it worked:

```
mamba --version
```

If you already have Anaconda or Miniconda, you can use `conda` in place of `mamba` in every command below. It solves the same environment more slowly.

2. Get the course package

Unpack the course package so that you have a directory containing `environment.yml`, `notebooks/`, `scripts/` and `config/`. That directory is referred to below as the package root. Move into it:

```
cd /path/to/the/package/root
ls environment.yml notebooks scripts config
```

All four names should print. If any is missing you are in the wrong directory.

3. Create the environment

```
mamba env create -f environment.yml
mamba activate CFDE_lung_env
```

The build takes several minutes. When it finishes, your shell prompt shows the environment name. Confirm the interpreter is the one you just built:

```
python --version
```

This must print Python 3.11.

4. Register the kernel for JupyterLab

JupyterLab lists kernels, not environments. Registering one gives you a named entry you can select inside a notebook, and makes it obvious when you have the wrong interpreter:

```
python -m ipykernel install --user --name cfde_lung_env --display-name "Python (CFDE_lung_env)"
```

The environment is `CFDE_lung_env`; the kernel identifier is lowercase by convention. Both names refer to the same interpreter.

Alternative: pip and venv

A plain virtual environment works if you cannot install conda:

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

`requirements.txt` carries the same 27 pins as `environment.yml`. The conda path is recommended because conda-forge supplies prebuilt binaries for the graph libraries used in clustering, which otherwise have to compile on your machine.

4. Starting JupyterLab in your browser

Start JupyterLab from the package root, with the environment active:

```
mamba activate CFDE_lung_env
cd /path/to/the/package/root
jupyter lab
```

A browser tab opens at an address on [localhost](#). If it does not open by itself, copy the URL printed in the terminal, including the token after `?token=`, and paste it into a browser.

Three things to check on the first notebook you open.

Kernel. The kernel name appears at the top right of the notebook. It must read [Python \(CFDE_lung_env\)](#). If it says anything else, use the Kernel menu, Change Kernel, and select it. A wrong kernel is the most common cause of an import error in an environment that was installed correctly.

Working directory. Start JupyterLab from the package root so that relative paths such as [config/paths.yaml](#) resolve. Each notebook also repairs this itself in its setup cell:

```
MODULE_ROOT = Path.cwd().resolve()
if MODULE_ROOT.name == "notebooks":
    MODULE_ROOT = MODULE_ROOT.parent
```

The cell walks up from wherever the kernel started until it finds a directory containing `scripts/`, then changes into it and puts it on the import path. That is why `from scripts.common.paths import load_config` works from either location.

Cell order. Run cells top to bottom with Shift and Enter. The notebooks carry state between cells, so running them out of order produces errors that look like data problems. After any failure, use Kernel, Restart Kernel and Run All Cells.

To stop the server, return to the terminal and press Ctrl and C twice.

5. The data this course uses

The course does not redistribute primary data. You download it yourself from four source portals, HuBMAP, GTEx, GEO and NASA OSDR, into [data/source/](#). **Discovery and download are the first part of Module 1**, which walks the portals, retrieves the files and verifies them. This section states what the course uses so you can budget disk and time; Appendix B is the full file list, totaling 814.87 MB across 20 files.

The teaching cohort is four HuBMAP lung donors. Each contributes one snRNA block, and Donor_2 also contributes the SNARE-seq2 multiome that Module 3 interprets:

Donor	Age	Sex	snRNA block	Also contributes
Donor_1	37	Male	HBM347.VDHS.379	
Donor_2	25	Female	HBM473.NKMR.872	SNARE-seq2 multiome HBM828.GPVG.252 (Module 3)
Donor_3	56.8	Female	HBM484.RGWP.797	
Donor_4	52.96	Male	HBM379.QCJT.435	

Three further resources enter at Module 4: GTEx v10 lung, GEO series GSE150910, and NASA OSDR study OSD-248. Appendix B names every file from all four.

6. Checking your setup

Run this before Module 1. It checks the interpreter, every pin, and the memory and disk available. Save it as `check_setup.py` in the package root and run `python check_setup.py`, or paste it into an empty notebook cell. Module 1 checks the data itself, once you have downloaded it.

```
import shutil
import sys
from importlib.metadata import PackageNotFoundError, version
from pathlib import Path

ROOT = Path.cwd().resolve()

print(f"python {sys.version.split()[0]} (course requires 3.11)")

pins = [
    line.strip()
    for line in (ROOT / "requirements.txt").read_text(encoding="utf-8").splitlines()
    if line.strip() and not line.startswith("#")
]
bad = []
for pin in pins:
    name, _, wanted = pin.partition("==")
    try:
        found = version(name)
    except PackageNotFoundError:
        found = None
    if found != wanted:
        bad.append((name, wanted, found or "not installed"))
print(f"packages: {len(pins) - len(bad)} of {len(pins)} match the pinned version")
for name, wanted, found in bad:
    print(f" MISMATCH {name}: want {wanted}, found {found}")

try:
    import os
    ram_gb = os.sysconf("SC_PAGE_SIZE") * os.sysconf("SC_PHYS_PAGES") / 1e9
    print(f"total RAM: {ram_gb:.1f} GB (16 recommended, 8 minimum)")
except (AttributeError, ValueError, OSError):
    print("total RAM: could not measure on this platform")

free_gb = shutil.disk_usage(ROOT).free / 1e9
print(f"free disk: {free_gb:.1f} GB (5 recommended)")
```

Every line should pass before you open Module 1. A package mismatch is worth fixing now: it is far cheaper than tracing a number that does not match the document three modules later.

7. How the course code is organized

Six directories carry the course. The division between them is the single most useful thing to understand before opening a notebook.

Directory	Holds	You will
config/	paths.yaml, every file path and analysis parameter	Read it, and edit it if you change a threshold
notebooks/	The four analysis notebooks	Run these
scripts/common/	Helpers shared by every module: paths, input and output, plotting, timing, statistics	Read these to see how a step is done
scripts/nb01_discovery/ through scripts/nb06_crossres/	The analysis itself, one package per stage	Read these for what a module actually computes
outputs/	Tables, figures and run parameters written by the notebooks	Inspect these; never edit them by hand
docs/	The module documents in Word format	Read alongside the notebooks

Learner module numbers and the internal script package names differ on purpose, because two script packages can serve one module:

Learner module	Notebook	Scripts
0	(no notebook)	Environment setup and code orientation
1	notebooks/01_discovery_and_qc.ipynb	nb01_discovery, nb02_qc
2	notebooks/02_integration_de_pathways.ipynb	nb03_integration, nb04_de
3	notebooks/03_multiome.ipynb	nb05_multimodal
4	notebooks/04_cross_ecosystem.ipynb	nb06_crossres
5	(no notebook)	Communication and assessment

Config blocks in `config/paths.yaml` use the learner-facing numbers, `module1` to `module4`.

The rule the package follows: **notebooks orchestrate, scripts compute, the config holds every path and parameter, and outputs are written rather than typed**. A notebook cell tends to be three to nine lines, an import and a call. The work is one level down.

This is why the notebooks look short. It is also the one thing that can make the course feel opaque on a first pass, so read a module in two passes. First run the notebook and see the shape of the argument. Then, for the step you care about, open the function it called.

Module 1 traced end to end, as an example:

```
notebooks/01_discovery_and_qc.ipynb, cell 10
  calls write_verification_outputs(cfg)
    scripts/nb01_discovery/verify.py, line 187
      calls verify_teaching_files(cfg)
        scripts/nb01_discovery/verify.py, line 32
          calls snrna_block_dir(cfg, processed, donor)
            scripts/common/paths.py, line 164
              reads paths from config/paths.yaml
```

Four steps from a notebook cell to the configuration file. Every module has the same depth.

8. Python patterns used throughout the course

Ten patterns account for nearly all the code you will read. Each is shown as it appears in the package, abridged where a full function would not fit.

1. *Configuration instead of constants*

```
def load_config(config_path: Path | None = None) -> dict[str, Any]:
    path = Path(config_path) if config_path else CONFIG_PATH
    with path.open() as fh:
        cfg = yaml.safe_load(fh)
    cfg["_module_root"] = MODULE_ROOT
    return cfg

def resolve(cfg: dict[str, Any], key: str) -> Path:
    """Resolve a paths.<key> entry to an absolute Path."""
    rel = cfg["paths"][key]
    return module_root(cfg) / rel
```

No file path is written twice anywhere in the package. Functions receive `cfg` and ask it for what they need. To point the course at different data you edit `config/paths.yaml` and change no code. This is the first thing to look for in any analysis codebase: if paths are typed into the analysis, the analysis only runs on the machine that wrote it.

2. *Paths as objects, not strings*

```
path = root / "snRNAseq" / bid
if path.exists():
    return path
```

`pathlib.Path` joins with `/`, so a path reads left to right in the order of the directories. `.exists()`, `.name`, `.parent` and `.stat().st_size` come free, and the same code runs on macOS, Linux and Windows. Nothing in the course concatenates path strings by hand.

3. *Type hints on every signature*

```
def sha256_optional(path: Path | str | None) -> str | None:
```

The signature says a caller may pass a `Path`, a string, or nothing, and may get back a string or nothing. Python does not enforce this at runtime, so it is documentation that your editor can check. The line `from __future__ import annotations` at the top of each file is what allows the modern `X | None` spelling on Python 3.11.

4. *Build rows as dictionaries, make the table once*

```
rows = []
for item in teaching_inputs(cfg):
    for name in expected:
        path = block_dir / name
        rows.append(
            {
                "donor_label": donor,
                "file_name": name,
                "required": True,
                "exists": path.exists(),
                "size_mb": round(path.stat().st_size / 1e6, 2) if path.exists() else None,
            }
        )
return pd.DataFrame(rows)
```

Appending dictionaries in a loop and building the DataFrame once at the end is both faster than growing a DataFrame row by row and far easier to read. One place decides the column names, and every row is guaranteed to have the same keys.

5. *Separate computing from saving*

```
def verify_teaching_files(cfg: dict[str, Any]) -> pd.DataFrame:
    ...
    return pd.DataFrame(rows)

def write_verification_outputs(cfg: dict[str, Any]) -> dict[str, Path]:
    verify = verify_teaching_files(cfg)
    verify.to_csv(tables / "module1_file_verification.tsv", sep="\t", index=False)
```

The function that computes returns a value and writes nothing. A second, thin function writes it to disk. You can call the first in a notebook and look at the result without producing a file, and you can test it without a filesystem. Almost every analysis package in the course splits this way, which is why the `write_*` functions are so short.

6. *Fail loudly, and say what to do*

```
# Never continue unlabeled. cell_class would silently become
# "unlabeled" for every nucleus and Modules 2 and 4 would report
# plausible but wrong composition.
raise FileNotFoundError(
    f"Azimuth labels unavailable for {label}: local secondary_analysis "
    f"({join_info.get('reason')}) and cache ({cache_info.get('reason')}). "
    "Build the cache with write_azimuth_label_cache(load_config()), "
    "or set module1_join_azimuth_labels: false to run without labels."
)
```

An exception that names the missing thing and the action to take is a feature, not an inconvenience. The alternative here is worse than a late failure. Had this returned without labels, every nucleus would have been classed `unlabeled`, and Modules 2 and 4 would have completed and reported a cell composition that looks entirely plausible and is wrong. A result that is silently incorrect costs far more than a run that stops. Notebook 01 applies the same rule with an assertion after the download check, so a missing file halts the run at its cause rather than surfacing three modules later.

7. *Derive a name, do not search for it*

```
# The block metadata file is named for the primary block ID. Derive it rather than
# globbing: a download manifest must name a file the learner does not have yet, and
# a glob can only ever find files that are already on disk.
metadata_name = tsvs[0].name if tsvs else (f"{primary}.tsv" if primary else None)
```

This one is worth pausing on, because the reasoning generalizes well beyond file names. Listing a directory tells you what you have. A plan has to be able to name what you do not have yet. Any time code discovers its own inputs by scanning, ask whether it is producing a plan or an inventory.

8. *Validate the configuration, do not trust it*

```
def _assert_no_decoy_keys(cfg: dict[str, Any]) -> None:
    for mod, forbidden in _FORBIDDEN_MODULE_KEYS.items():
        bad = sorted(forbidden.intersection(cfg.get(mod) or {}))
        if bad:
            raise ValueError(
                f"config/{mod} contains removed decoy key(s) {bad}; use live knobs instead"
            )
```

A configuration key that no longer controls anything is more dangerous than a missing one, because someone will change it and expect a different result. Checking for dead keys at load time turns a silent no-op into an error at the moment the config is read.

9. Import heavy libraries lazily

```
def read_h5ad(path: Path | str, backed: str | None = None):
    """Lazy import scanpy/anndata to keep discovery notebooks light."""
    import scanpy as sc

    return sc.read_h5ad(path, backed=back)
```

`import scanpy` is slow and pulls in a large dependency tree. Putting it inside the one function that needs it means a notebook that only inspects file metadata starts immediately. Use this sparingly: the cost is that an import error surfaces at call time rather than at startup.

10. Record what produced the number

```
def sha256_file(path: Path | str, *, chunk: int = 1 << 20) -> str:
    """Hex sha256 of a file on disk."""
    h = hashlib.sha256()
    with Path(path).open("rb") as fh:
        while True:
            block = fh.read(chunk)
            if not block:
                break
            h.update(block)
    return h.hexdigest()
```

Every module writes a `*_run_params.json` recording its parameters, package versions, wall time and peak memory, and the file inventory records a SHA-256 for each input. The same principle governs the documents you are reading: values are read from the table that produced them rather than typed in.

```
def _value_from_table(path: Path, key: str) -> str:
    df = pd.read_csv(path, sep="\t")
    if key not in df.columns:
        raise KeyError(f"Column {key} not in {path.name}")
    return str(df[key].dropna().iloc[0])
```

A number in a report that was copied by hand is a number that can drift from the analysis without anyone noticing. Reading it from the artifact removes the possibility.

9. Errors you are likely to meet

What you see	What it means	What to do
<code>ModuleNotFoundError: No module named 'scripts'</code>	The kernel is not running from the package root	Run the notebook setup cell, or restart JupyterLab from the package root
<code>Cannot find scripts/ from cwd=...</code>	The notebook was copied outside the package	Open it from <code>notebooks/</code> inside the package
<code>ModuleNotFoundError: No module named 'scanpy'</code>	The wrong kernel is selected	Kernel, Change Kernel, Python (CFDE_lung_env)
<code>AssertionError: Missing required files in Module 1</code>	Part of the 814.87 MB is not downloaded, often the Module 4 files	Complete the download listed in Appendix B, then rerun the cell

<code>FileNotFoundError</code> naming a <code>raw_expr.h5ad</code> path	The file is missing, or the folder is named by the processed ID	Rename the folder to the primary block ID from the table above
The kernel restarts during Module 2	The machine ran out of memory at the differential expression step, which peaks at 7.6 GB	Close other applications, or run on a machine with 16 GB
The first analysis cell hangs for a minute	Numba is compiling on first use inside scanpy	Wait once. Later runs are fast

Appendix A: Pinned packages

Read from `requirements.txt` in this package at build time.

Package	Version
numpy	2.4.6
pandas	2.3.3
scipy	1.17.1
matplotlib	3.11.1
seaborn	0.13.2
PyYAML	6.0.3
h5py	3.16.0
tables	3.11.1
tqdm	4.70.0
anndata	0.12.19
scanpy	1.11.5
leidenalg	0.12.0
igraph	1.0.0
umap-learn	0.5.12
scikit-learn	1.9.0
harmonypy	2.0.0
gseapy	1.3.1
scvelo	0.3.4
session-info	1.0.1
mudata	0.3.10
muon	0.1.9
pydeseq2	0.5.4
jupyterlab	4.6.2
ipykernel	7.3.0
ipywidgets	8.1.8
nbformat	5.10.4
nbclicent	0.11.0

Appendix B: Required downloads

Every file the course needs under [data/source/](#), read from the package manifest at build time: 20 files totaling 814.87 MB. Download all of them before you open Module 1, including the GTEx, GEO and OSDR files that only Module 4 uses.

donor_label	file_name	size_mb	role	required	exists
Donor_1	raw_expr.h5ad	68.57	Module 1 QC matrix	True	True
Donor_1	HBM347.VDHS.379.tsv	0.02	block metadata	True	True
Donor_2	raw_expr.h5ad	104.55	Module 1 QC matrix	True	True
Donor_2	HBM473.NKMR.872.tsv	0.02	block metadata	True	True
Donor_3	raw_expr.h5ad	150.92	Module 1 QC matrix	True	True
Donor_3	HBM484.RGWP.797.tsv	0.02	block metadata	True	True
Donor_4	raw_expr.h5ad	70.56	Module 1 QC matrix	True	True
Donor_4	HBM379.QCJT.435.tsv	0.02	block metadata	True	True
Donor_2	multiome_mofa.hdf5	140.78	Module 3 MOFA interpretation (default path)	True	True
GTEx	gene_tpm_v10_lung.gct.gz	77.6	Module 4 GTEx lung TPM (levels and HuBMAP bridge)	True	True
GTEx	gene_reads_v10_lung.gct.gz	32.49	Module 4 GTEx lung counts (aging DE)	True	True
GTEx	GTEx_Analysis_v10_Annotations_SampleAttributesDS.txt	38.5	Module 4 GTEx sample metadata	True	True
GTEx	GTEx_Analysis_v10_Annotations_SubjectPhenotypesDS.txt	0.02	Module 4 GTEx subject metadata (age brackets)	True	True

GEO	GSE150910_gene-level_count_file.csv.gz	8.12	Module 4 IPF vs control counts	True	True
GEO	GSE150910_series_matrix.txt.gz	0.01	Module 4 GEO series and sample metadata	True	True
OSDR	GLDS- 248_rna_seq_RSEM_Unnormalized_Counts_rRNArm_GLbulkRNAseq.csv	10.9	Module 4 RSEM counts (rRNArm) for the flight contrast	True	True
OSDR	GLDS- 248_rna_seq_differential_expression_rRNArm_GLbulkRNAseq.csv	111.54	Module 4 GeneLab DE table (independen t cross- check)	True	True
OSDR	GLDS-248_rna_seq_contrasts_GLbulkRNAseq.csv	0.01	Module 4 GeneLab contrast definitions	True	True
OSDR	s_OSD-248.txt	0.06	Module 4 ISA sample metadata (arm assignment)	True	True
OSDR	a_OSD-248_transcription-profiling_rna-sequencing-(rna-seq)_Illumina NovaSeq 6000.txt	0.16	Module 4 ISA assay metadata (library prep, sequencer)	True	True

6 further columns in the source file. Source: module1_learner_download_manifest.tsv

References

Software documentation for the tools this module installs.

Tool	Documentation
Miniforge (conda-forge installer)	https://github.com/conda-forge/miniforge
conda-forge	https://conda-forge.org
JupyterLab	https://jupyterlab.readthedocs.io
Python 3.11 tutorial	https://docs.python.org/3.11/tutorial/
pandas	https://pandas.pydata.org/docs/
scanpy and AnnData	https://scanpy.readthedocs.io

Two files in the package carry everything this module refers to: [config/paths.yaml](#) for every path and analysis parameter, and [requirements.txt](#) for the pinned environment.